8.16 File handling

If you have data in a program, when you stop or close a program all of that data is lost. If you need to save data to use again later then you need to save it externally into a text file. The storage and access of data in a file is called **file handling**.

KEY WORD

file handling: programming statements that allow text files to be opened, read from, written to and closed.

COMPUTER SCIENCE IN CONTEXT

When playing computer games, for example, on an X-Box or PlayStation, you are able to save the game and then continue from the same point next time. It does this by saving data about your current progress in a file. When your program saves, this data is updated; it might store your current position, health, points, money, etc. When you restart the program, it goes to this file and loads the data. This lets you start playing at the exact point you left it.

Reading from a file

You need to be able to read a single item of data, or a single line of text. This means all of the data will be on the first line in the text file, so you do not need to check how many lines of text are in the file. Once you have read in the data you can then manipulate it, for example, if it is a line of text you can split it into individual words, or use it as one item of data.

To read a value from a file you need to:

- Open the file using its filename (the filename will be a string value).
- Read the data value, and do something with it (e.g. output it, store it in a variable).
- Close the file.

You can use the pseudocode commands:

```
OPEN filename
variable identifier ← READ(filename)
CLOSE filename
```

Example 1

Reading and outputting a word stored in the text file `data.txt`:

```
OPEN "data.txt"
OUTPUT(READ("data.txt"))
CLOSE "data.txt"
```

VB.NET

```
Dim theFile As New System.IO.StreamReader("data.txt")
Console.WriteLine(theFile.ReadLine())
theFile.Close()
```

Python

```
theFile = open("data.txt", 'r')
print(theFile.read())
theFile.close()
```

Java

```
public static void main(String args[]){
    try{
        FileReader f = new FileReader("data.txt");
        BufferedReader reader = new BufferedReader(f);
        System.out.println(reader.readLine());
        reader.close();
    }catch(Exception e){
        System.err.println("No file");
    }
}
```

Example 2

Read and output the number stored in the file `myNumber.txt` by storing the filename in a variable:

```
Filename ← "myNumber.txt"
OPEN Filename
TheFileData ← READ(Filename)
CLOSE(Filename)
OUTPUT(TheFileData)
```

VB.NET

```
Dim filename As String = "myNumber.txt"
Dim theFileData As String
Dim theFile As New System.IO.StreamReader(filename)
theFileData = theFile.ReadLine()
theFile.Close()
Console.WriteLine(theFileData)
```

Python

```
filename = "myNumber.txt"
theFile = open(filename, 'r')
theFileData = theFile.read()
theFile.close()
print(theFileData)
```

Java

```
public static void main(String args[]){
    String filename = "myNumber.txt";
    String theFileData;
    try{
        FileReader f = new FileReader(filename);
        BufferedReader reader = new BufferedReader(f);
        theFileData = reader.readLine();
        reader.close();
    }catch(Exception e){
        System.err.println("No file");
    }
}
```

Writing to a file

The specification states that you will only need to write a single item of data or a line of text. This means you will be overwriting any data that is already in the file, you do not need to worry about appending (to add onto the end) to data that already exists, or writing multiple values to the same file.

To write a value to a file you need to:

- Open the file using its filename (the filename will be a string value).
- Write the data to the file.
- Close the file.

You can use the pseudocode commands:

```
OPEN filename
WRITE data
CLOSE filename
```

Example 1

Write the word `"red"` to the file `colour.txt`:

```
OPEN "colour.txt"
WRITE "red"
CLOSE "colour.txt"
```

VB.NET

```
Dim fileWrite As New System.IO.StreamWriter("colour.txt")
fileWrite.WriteLine("red")
fileWrite.Close()
```

Python

```
fileData = open("colour.txt",'w')
fileData.writelines("red")
fileData.close()
```

Java

```
public static void main(String args[]){
    try{
        FileWriter f = new FileWriter("colour.txt");
        BufferedWriter out = new BufferedWriter(f);
        out.write("red");
        out.close();
    }catch(Exception e){
        System.err.println("No file");
    }
}
```

Example 2

Write the number `100` to the file `myData.txt` storing the filename in a variable:

```
Filename ← "myData.txt"
OPEN Filename
WRITE 100
CLOSE Filename
```

VB.NET

```
Dim filename As String = "myData.txt"
Dim fileWrite As New System.IO.StreamWriter(filename)
fileWrite.WriteLine(100)
fileWrite.Close()
```

Python

```
filename = "myData.txt"
fileData = open(filename,'w')
fileData.writelines(100)
fileData.close()
```

Java

```
public static void main(String args[]){
        try{
        String filename = "myData.txt";
        FileWriter f = new FileWriter(filename);
        BufferedWriter out = new BufferedWriter(f);
        out.write(100);
        out.close();
    }catch(Exception e){
        System.err.println("No file");
    }
}
```

PROGRAMMING TASK 8.5

A maths quiz game needs to keep a record of the highest number of points players has gained. The maths quiz is made of randomly generated questions; the mathematical operation and the numbers are all randomly generated. The user keeps on being given new questions until they get one wrong. The points equate to the number of questions they got correct.

Getting started

1. In pairs discuss how the program can randomly generate the numbers within reasonable bounds, e.g. between 1 and 20.
2. In pairs discuss how the program can randomly generate the symbol limited to + − / * ^.
3. In pairs discuss how the highest score can be stored in a file. Discuss when the file will be read from, and when it will be written to.

Practice

1. Write the program to randomly generate one maths question (random numbers and symbol). Output the question and ask the user for the answer. Check if it is correct and output an appropriate response.
2. Amend your program to repeatedly ask the user questions until they get one incorrect. Keep track of how many questions they get correct.
3. Amend your program so when the user gets a question incorrect, the current high score is loaded from a text file. Replace the high score if the user has more points.

Challenge

1. Text files can be read one line at a time. Find out how to store more than one high score (e.g. a top-ten) in a file, and rearrange the high-score table when a user gains a score worthy of including.

TIP

There are five options so a number could represent each symbol.

REFLECTION

Consider the program you made for Programming Task 8.5. How did you decide which stage(s) to complete and when to stop? How did you find example code that you needed? Was this an efficient method of finding the required statements?

How did you test your program during development? Did you test the program after each line of code, or did you write a section? Did this method work for the program? Would you do it the same way in the future?

How did you test your program once it was complete? Did you set out a structured test plan with different types of data? Did you ask other people to help you test it? Was your chosen method appropriate for the program, or did you have to change it during the testing process?

Questions

43 Why do some programs need to store data in files?

44 What are the **three** stages that need to be followed to write data to a file?

45 Write a program to read in a value from the file `dataStore.txt` and output it.

46 Write a program to ask the user to input a filename, then store the word `"house"` in the file.

SUMMARY

A variable is a space in memory, with an identifier, that can store a data item that can change while the program is running.
A constant is a space in memory, with an identifier, that can store a data item that cannot change while the program is running.
Integer data type stores whole numbers. Real data type stores decimal numbers. Char data type stores a single character. String data type stores a series of characters. Boolean data type stores True or False.
Input allows the user to enter data into a system.
Output allows the program to display data to the user.
There are three constructs in programming; sequence, selection and iteration.
There are two types of selection; IF and CASE.
IF statements can include ELSEIF and ELSE.
There are three types of iteration; count-controlled loops (a set number of iterations), pre-condition loops (condition is tested before starting the loop) and post-condition loops (condition is tested after completing the code in the loop).
Totalling requires initialising the total to 0, then adding the values to it.
Counting requires initialising the count to 0, then adding 1 to it.
Nested statements are when selection/iteration are within a selection/iteration construct.
Subroutines are self-contained code, with an identifier that can be called from elsewhere in the program.
Subroutines reduce repeated code.
Subroutines can be procedures (that do not return a value) or functions (that return a value).
A parameter is a value sent to a subroutine.
Library routines contain pre-written and pre-tested subroutines that can be used in a program.
A maintainable program includes meaningful identifiers, addition of comments and subroutines.
An array allows a set of data, of the same data type, to be stored under one identified.
Each element in an array has an index. This might start at 0 or 1 depending on your language.